

LaRT v2.00 User Manual

Contents

1. Introduction	3
2. Installation and Build	3
2.1 Prerequisites	3
2.2 Compilation	3
2.3 Running	4
3. Input File Format	4
4. Grid Configuration	4
4.1 Cartesian Grid	4
4.2 Grid Geometry	5
4.3 Symmetry and Periodic Boundaries	5
4.4 Distance Units	5
4.5 Grid Type Selection	6
5. Source Configuration	6
5.1 Source Geometry	6
5.2 Spectral Type	7
5.3 Line Identification	7
5.3.1 Combined Hydrogen + Deuterium Ly α	8
5.4 Frequency and Wavelength Grid	9
6. Medium Properties	10
6.1 Optical Depth Normalization	10
6.2 Temperature	10
6.3 Density	11
6.4 Velocity Fields	11
7. AMR Grid Mode	12
7.1 Enabling AMR Mode	12
7.2 Generic AMR Data Format	12
7.3 Tau Normalization Convention	13
7.4 Source Geometries in AMR Mode	13

8. Clumpy Medium Mode	13
8.1 Overview	13
8.2 Enabling Clump Mode	13
8.3 Clump Parameters	14
8.4 Spatially-varying clump properties	15
8.5 Standalone clump generator (<code>make_clumps.x</code>)	15
8.6 Clump Velocity	16
9. Observers and Peel-off	16
9.1 Observer Placement	16
9.2 Multiple Observers	17
9.3 Image Parameters	17
9.4 Peel-off Output Flags	17
9.5 Sight-line Optical-depth and Column-density Maps	17
9.6 Inside Observer (HEALPix All-Sky)	19
10. Output Files	20
10.1 Primary Output Arrays	20
10.2 Angular Spectrum J_{μ}	21
10.3 Peel-off cube units and conversion to J_{μ} units	22
10.4 Output Control Parameters	23
11. Dust and Polarization	23
12. Parallelism	24
13. Complete Parameter Reference	24
14. Python Utilities	28
14.1 Reading FITS output: <code>read_lart.py</code>	28
15. Example Input Files	30
15.1 Uniform Sphere, $\text{Ly}\alpha$, Static	30
15.2 Sphere with Hubble Outflow and Peel-off	31
15.3 Realistic Galaxy (FITS Density Field, Stokes Polarization)	31
15.4 AMR Sphere (Generic Text Format)	32
15.5 Clumpy Sphere with Random Velocities	32

1. Introduction

LaRT (Lyman-alpha Radiative Transfer) is a Monte Carlo radiative transfer code for computing the propagation of resonance-line photons through astrophysical neutral-gas media. The default transition is Lyman- α ($\text{Ly}\alpha$, H I 1216 Å), but many other UV/optical/IR lines are supported (see §5.3). Dust scattering and absorption, full Stokes polarization tracking, Hubble-flow and other bulk velocity fields, and spatially varying density, temperature, and velocity from external data files are all supported.

Starting from v2.00, the Cartesian grid is supplemented by:

- **AMR mode** — octree adaptive mesh refinement, allowing spatially varying resolution (§7).
- **Clumpy medium mode** — a population of compact, optically thick spherical clumps embedded in a transparent inter-clump medium (§8).

This manual covers v2.00 and v2.10. The only user-visible change in v2.10 is the introduction of `par%grid_type` as a unified selector for the three grid modes; the old `par%use_amr_grid` and `par%use_clump_medium` flags remain valid for backward compatibility.

2. Installation and Build

2.1 Prerequisites

- Fortran 2003+ compiler: Intel `ifort/ifx` or GNU `gfortran` ≥ 7 .
- CFITSIO library.
- MPI library (MPICH or OpenMPI).

2.2 Compilation

```
cd LaRT_v2.00      # or LaRT_v2.10
make                # produces LaRT.x  (pure MPI)
make DEBUG=1        # full debug flags + runtime checks
make F90=mpif90      # override compiler
make clean           # remove .o and .mod files
make cleanall        # also remove executables
make ramses2fits     # auxiliary RAMSES converter
```

Every `make` invocation recompiles all objects (no incremental builds).

2.3 Running

```
mpirun -np 8 LaRT.x input.in
```

Output is written to `<base>.fits.gz`, where `<base>` is the input file base name (without `.in`) unless `par%out_file` is set explicitly.

3. Input File Format

Input files use Fortran *namelist* syntax. All parameters are prefixed with `par%` and collected in a single `¶meters` block.

```

1 &parameters
2   par%no_photons      = 1e6
3   par%temperature     = 1.0e4      ! gas temperature [K]
4   par%taumax          = 1.0e4
5   par%nx = 101, par%ny = 101, par%nz = 101
6   par%rmax            = 1.0
7   par%source_geometry = 'point'
8   par%spectral_type   = 'voigt'
9   par%nxfreq          = 121
10 /
```

- String values must be enclosed in single quotes.
- Multiple parameters may appear on one line, separated by commas.
- Lines beginning with `!` are comments.
- Unspecified parameters retain compiled defaults (listed in §13.).

4. Grid Configuration

4.1 Cartesian Grid

The simulation domain is a rectangular box centered at the origin. By default $x \in [-x_{\max}, +x_{\max}]$ (and similarly for y and z). The cell size is $\Delta x = 2x_{\max}/n_x$.

Parameter	Default	Description
par%nx, par%ny, par%nz	101	Number of cells per axis
par%xmax, par%ymax, par%zmax	1.0	Half-box size per axis (code units)
par%rmax	-999	Spherical boundary: density = 0 for $r > r_{\max}$
par%rmin	-999	Inner cavity: density = 0 for $r < r_{\min}$

If `par%input_field` or `par%dens_file` is given, (nx, ny, nz) and $(xmax, ymax, zmax)$ are read from the FITS header automatically.

4.2 Grid Geometry

`par%geometry` selects the spatial symmetry used for accumulating and saving the mean-intensity array J :

Value	J output geometry
'sphere' (default)	1-D radial profile $J(r)$
'cylinder'	2-D cylindrical $J(\rho, z)$
'rectangle'	3-D Cartesian cube $J(x, y, z)$
'plane_atmosphere'	1-D slab $J(z)$
'spherical_atmosphere'	1-D spherical shell $J(r)$

4.3 Symmetry and Periodic Boundaries

Parameter	Default	Effect
par%xyz_symmetry	.false.	Simulate only the $+x, +y, +z$ octant ($8\times$ RAM saving)
par%xy_symmetry	.false.	Simulate only the $+x, +y$ quadrant ($4\times$ RAM saving)
par%xy_periodic	.false.	Periodic in x, y : photons exiting one face re-enter the opposite

Symmetry options are disabled automatically when peel-off is active.

4.4 Distance Units

Parameter	Default	Description
par%distance_unit	"	Physical unit: 'kpc', 'pc', 'au', or "" (dimensionless)
par%distance2cm	1.0	Alternatively: 1 code unit in cm

In dimensionless mode (`par%distance_unit = ""`), the gas density is determined entirely by `par%taumax` and the grid geometry; no physical unit is needed.

4.5 Grid Type Selection

Parameter	Default	Description
<code>par%grid_type[†]</code>	<code>'car'</code>	v2.10+: <code>'car'</code> (Cartesian), <code>'amr'</code> , or <code>'clump'</code>
<code>par%use_amr_grid</code>	<code>.false.</code>	v2.00: enable AMR (deprecated in v2.10)
<code>par%use_clump_medium</code>	<code>.false.</code>	v2.00: enable clump mode (deprecated in v2.10)

5. Source Configuration

5.1 Source Geometry

`par%source_geometry` controls the spatial distribution of photon emission:

Value	Description
<code>'point'</code>	Point source at (<code>par%xs_point</code> , <code>par%ys_point</code> , <code>par%zs_point</code>); default (0,0,0)
<code>'uniform'</code>	Uniform volume emissivity throughout the grid (or inside <code>par%source_rmax</code>)
<code>'uniform_sphere'</code>	Uniform within a sphere of radius <code>par%source_rmax</code>
<code>'uniform_xy'</code>	Uniform in the $z = 0$ plane
<code>'gaussian'</code>	Gaussian: $\exp(-r^2/2\sigma_r^2)\exp(-z^2/2\sigma_z^2)$; widths <code>par%source_rscale</code> , <code>par%source_zscale</code>
<code>'exponential'</code>	Exponential disc: $\exp(-r/r_s)\exp(- z /z_s)$; scales <code>par%source_rscale</code> , <code>par%source_zscale</code>
<code>'exponential_sphere'</code>	Exponential clipped to sphere of radius <code>par%source_rmax</code>
<code>'exponential_cylinder'</code>	Exponential clipped to a cylinder
<code>'sersic' / 'ssh'</code>	Sérsic profile; index <code>par%sersic_m</code> , half-light radius <code>par%Reff</code>
<code>'diffuse_emissivity'</code>	3-D emissivity read from <code>par%emiss_file</code>
<code>'star_file'</code>	Individual point sources from text file <code>par%star_file</code> (columns: x, y, z, L)

Associated parameters:

Parameter	Default	Description
<code>par%xs_point, par%ys_point, par%zs_point</code>	0.0	Point source position (code units)
<code>par%source_rmax</code>	-999	Outer truncation radius for extended sources
<code>par%source_rscale</code>	-999	Radial scale length
<code>par%source_zscale</code>	0.0	Vertical scale height
<code>par%sersic_m</code>	4.0	Sérsic index
<code>par%Reff</code>	1.0	Effective (half-light) radius
<code>par%comoving_source</code>	<code>.true.</code>	Source photons co-moving with the local gas

5.2 Spectral Type

`par%spectral_type` sets the frequency distribution of emitted photons:

Value	Description
<code>'monochromatic'</code>	Delta function at line center ($x = 0$)
<code>'voigt'</code>	Voigt profile at the local gas temperature
<code>'voigt0'</code>	Voigt profile at the fixed temperature <code>par%temperature0</code>
<code>'gaussian'</code>	Gaussian with width <code>par%gaussian_width_vel</code> [km s^{-1}]
<code>'continuum'</code>	Flat spectrum over <code>[par%xfreq_min, par%xfreq_max]</code>
<code>'line_prof_file'</code>	Arbitrary profile read from <code>par%line_prof_file</code>

5.3 Line Identification

The atomic transition is selected with `par%line_id` (default: $\text{Ly}\alpha$).

par%line_id	Transition	Wavelength
(default)	H I Ly α	1215.67 Å
'ly_alpha_HD'	H I + D I Ly α	1215.67 Å (see below)
'CIV_1548'	C IV doublet	1548.19 / 1550.77 Å
'NV_1239'	N V doublet	1238.82 / 1242.80 Å
'OVI_1032'	O VI doublet	1031.91 / 1037.61 Å
'SiIV_1394'	Si IV doublet	1393.76 / 1402.77 Å
'NaI_D'	Na I D doublet	5891.58 / 5897.56 Å
'CaII_HK'	Ca II H & K doublet	3934.78 / 3969.59 Å
'MgII_2796'	Mg II doublet	2796.35 / 2803.53 Å
'AlII_1671'	Al II singlet	1670.79 Å
'SiII_1190' / 'SiII_1193'	Si II doublet (+ fluorescence)	1190.42 / 1193.29 Å
'SiII_1260'	Si II (+ fluorescence)	1260.42 Å
'SiII_1304'	Si II (+ fluorescence)	1304.37 Å
'SiII_1527'	Si II (+ fluorescence)	1526.71 Å
'FeII_2250'	Fe II (+ fluorescence)	2249.88 Å
'FeII_2261'	Fe II (+ fluorescence)	2260.78 Å
'FeII_UV3' / 'FeII_2344'	Fe II UV3 multiplet	2344.21 Å
'FeII_UV2' / 'FeII_2383'	Fe II UV2 multiplet	2382.76 Å
'FeII_UV1' / 'FeII_2600'	Fe II UV1 multiplet	2600.17 Å
'HeI_10833'	He I triplet	10832.06 Å

The numerical `line%line_type` that controls the dispatch of `par%calc_voigt` / `par%do_resonance` (set internally by `setup_resonance_line`) takes the following values: 1 = singlet (one upward transition); 2 = doublet (two upward transitions sharing a single thermal width, e.g. C IV, N V, O VI, Na I D, Ca II H&K, Mg II, Si IV); 4 = one resonance plus one or more fluorescent decays (most Si II, Fe II UV3); 5 = multiple upward + multiple downward transitions, the most general case (Si II 1190/1193, Fe II UV1, Fe II UV2); 6 = three upward transitions plus one downward transition (He I 10833); 7 = combined H + D Ly α (ly_alpha_HD, two coexisting two-level resonance scatterers, no fine structure).

5.3.1 Combined Hydrogen + Deuterium Ly α

The line id 'ly_alpha_HD' treats neutral hydrogen and deuterium Ly α as two coexisting two-level resonance scatterers in the same medium. The two transitions sit only 0.33 Å (~ 82 km/s) apart and have nearly identical atomic constants except for the atomic mass and rest wavelength. The deuterium-to-hydrogen number-abundance ratio is set by

Parameter	Default	Description
<code>par%D_to_H_ratio</code>	1.5×10^{-5}	n_D/n_H (cosmic primordial)
<code>par%include_deuterium</code>	<code>.false.</code>	Convenience flag

Setting `par%include_deuterium = .true.` together with `par%line_id = 'ly_alpha'` promotes the run automatically to `'ly_alpha_HD'`, so existing input files can opt in without changing the line id.

Physical assumptions in this version (v1):

- Both H and D are treated as simple two-level resonance scatterers (`line_type=7`). Fine-structure splitting is *not* included for either species; if `par%fine_structure = .true.` is set together with `'ly_alpha_HD'`, the code emits a warning and disables fine structure.
- The phase function is the simple dipole ($E_1 = 1$, $E_2 = 0$, $E_3 = 1$), identical for both species.
- Oscillator strengths and damping rates are taken to be equal for H and D ($f_D = f_H$, $\Gamma_D = \Gamma_H$ to better than 0.1%). Mass and rest wavelength differences are fully accounted for in the Voigt evaluation and species selection.
- Photon `xfreq` is carried in H-frame Doppler units throughout. D-frame conversions happen on-the-fly in the combined Voigt and scattering routines via constants precomputed at line setup (`line%delta_nu_HD_Hz`, `line%ratio_Dfreq_HD`, `line%ratio_voigta_HD`). No per-cell deuterium arrays are stored.
- During a deuterium scattering event, the perpendicular thermal velocity components and the recoil shift are evaluated using the H atomic constants, which produces a small ($\sim 1/\sqrt{2}$) error in the re-emission shape from D-scattered photons. Because D scatters are rare at cosmic abundance and the dominant D feature (the absorption dip) depends on opacity rather than re-emission shape, this is acceptable for v1. Per-photon species tracking is also not yet implemented.

The Deuterium Ly α rest wavelength used is $\lambda_D = 1215.337$ (NIST vacuum), ~ 0.33 blueward of $\lambda_H = 1215.668$. In LaRT's velocity convention ($v = -c \Delta\lambda/\lambda_H$, so negative v is blueshifted), the D Ly α line center sits at $v_D \approx -82$ km/s.

A reference example with five validation runs and a Jupyter notebook is provided under `examples/lya_HD/` (see `run.sh` and `inspect_HD.ipynb`). One of those runs is configured for direct comparison against Dijkstra, Haiman & Spaans (2006), ApJ 649, 14, Figure 3 ($D/H = 3 \times 10^{-5}$, $\tau \sim 10^6$).

5.4 Frequency and Wavelength Grid

Internally, photon frequencies are tracked in the dimensionless Doppler variable:

$$x = \frac{\nu - \nu_0}{\Delta\nu_D}, \quad \Delta\nu_D = \frac{\nu_0}{c} \sqrt{\frac{2kT}{m}}, \quad (1)$$

where ν_0 is the line-center frequency, T is the local gas temperature, and m is the atomic/ionic mass. Output spectra are binned onto a uniform grid in x by default. A wavelength or velocity grid can be requested instead:

Parameter	Default	Description
<code>par%nxfreq</code>	121	Number of spectral bins
<code>par%xfreq_min, par%xfreq_max</code>	auto	Limits of x -grid
<code>par%xfreq0</code>	0.0	Frequency offset applied to all emitted photons
<code>par%nwavelength</code>	0	Number of wavelength bins (overrides x -grid)
<code>par%wavelength_min, par%wavelength_max</code>	—	[Å]
<code>par%nvelocity</code>	0	Number of velocity bins (overrides x -grid)
<code>par%velocity_min, par%velocity_max</code>	—	[km s ⁻¹]

The range is set automatically from the mean medium temperature when no limits are provided.

6. Medium Properties

6.1 Optical Depth Normalization

The overall opacity is scaled by one of the following (specify at most one):

Parameter	Description
<code>par%taumax</code>	Line-center τ_0 from the grid center to the $+z$ boundary along the z -axis
<code>par%tauhomo</code>	Same, assuming a uniform density field (scales an inhomogeneous field proportionally)
<code>par%N_HI_max</code>	H I column density from center to $+z$ boundary [cm ⁻²]
<code>par%N_HI_homo</code>	Same, assuming uniform density

For AMR grids, `par%taumax` is the pole-to-face optical depth (box center $\rightarrow +z$ face along z), consistent with the Cartesian convention for a sphere of radius $R = L/2$.

6.2 Temperature

Parameter	Default	Description
<code>par%temperature</code>	10 ⁴ K	Uniform gas temperature
<code>par%temperature0</code>	$= T$	Temperature for <code>spectral_type='voigt0'</code> source
<code>par%bturb</code>	0.0	Turbulent velocity dispersion added in quadrature to thermal width [km s ⁻¹]
<code>par%temp_file</code>	"	3-D temperature FITS cube [K]

6.3 Density

Analytic profiles. Without external files, the density is set analytically via the opacity scale parameters (`par%taumax` etc.) combined with an optional radial profile:

Profile type	Key parameter	Functional form
Uniform (sphere or box)	<code>par%rmax</code>	Constant n_{H} inside $r < r_{\text{max}}$
Spherical shell	<code>par%rmin, par%rmax</code>	Constant n_{H} between r_{min} and r_{max}
Exponential	<code>par%density_rscale</code>	$n \propto \exp(-r/r_s)$

External FITS data.

Parameter	Description
<code>par%dens_file</code>	3-D H I number density cube [cm^{-3}]
<code>par%temp_file</code>	3-D temperature cube [K]
<code>par%velo_file</code>	4-D velocity cube (v_x, v_y, v_z) [km s^{-1}]
<code>par%emiss_file</code>	3-D emissivity cube [$\text{photons s}^{-1} \text{cm}^{-3}$]
<code>par%input_field</code>	Base name: reads <code><base>.dens.fits.gz, .temp., .velo.</code>

If `par%taumax` (or `par%N_HI_max`) is specified alongside a density file, the density is rescaled so that the target optical depth is achieved.

6.4 Velocity Fields

`par%velocity_type` selects the bulk velocity model. The default (empty string) is a static medium.

Value	Key parameters	Velocity law
'hubble'	<code>par%Vexp</code>	$\mathbf{v}(r) = V_{\text{exp}}(r/r_{\text{max}})\hat{r}$; $V_{\text{exp}} > 0$: outflow
'constant_radial'	<code>par%Vexp</code>	$\mathbf{v}(r) = V_{\text{exp}}\hat{r}$ (constant magnitude)
'parallel_velocity'	<code>par%Vx, par%Vy, par%Vz</code>	Uniform $\mathbf{v} = (V_x, V_y, V_z)$ [km s^{-1}]
'ssh'	<code>par%Vpeak, par%rpeak</code>	Bilinear profile (Song, Seon & Hwang 2020)
'rotating_solid_body'	<code>par%Vrot, par%risol</code>	Solid-body rotation
'rotating_galaxy_halo'	<code>par%Vrot, par%q</code>	Galaxy halo rotation curve

The 'ssh' profile rises linearly to V_{peak} at r_{peak} , then shifts by ΔV toward r_{max} :

$$v(r) = \begin{cases} V_{\text{peak}} r/r_{\text{peak}} & r < r_{\text{peak}} \\ \left[V_{\text{peak}} + \Delta V \frac{r-r_{\text{peak}}}{r_{\text{max}}-r_{\text{peak}}} \right] \hat{r} & r \geq r_{\text{peak}}. \end{cases} \quad (2)$$

A velocity FITS file (`par%velo_file`) may be combined with a systematic `par%velocity_type`.

Other velocity-related parameters:

Parameter	Default	Description
par%Omega	28.0	Shear rate [(km s ⁻¹) kpc ⁻¹] for TIGRESS data
par%recoil	.false.	Include photon-recoil frequency shift in scattering
par%core_skip	.false.	Enable core-skipping acceleration algorithm

7. AMR Grid Mode

7.1 Enabling AMR Mode

v2.00:

```

1 par%use_amr_grid = .true.
2 par%amr_type      = 'generic'    ! always specify explicitly
3 par%amr_file      = 'grid.dat'
```

v2.10 (preferred):

```

1 par%grid_type = 'amr'
2 par%amr_type  = 'generic'
3 par%amr_file  = 'grid.dat'
```

Parameter	Default	Description
par%amr_type	'ramses'	Format: 'generic' (text) or 'ramses' (binary snapshot)
par%amr_file	"	Path to AMR data file or RAMSES snapshot directory
par%amr_snapnum	1	Snapshot number for RAMSES output

Note: the compiled default for par%amr_type is 'ramses'. Always specify par%amr_type = 'generic' explicitly when using the text format.

7.2 Generic AMR Data Format

Plain text: a header followed by one line per leaf cell.

```

1 # comment lines start with #
2 BOXLEN 100.0          ! box side length in code units
3 NLEAF   8192          ! number of leaf cells
4 ! columns: x  y  z  level  nH[cm^-3]  T[K]  vx[km/s]  vy[km/s]  vz[km/s]
5 50.5  50.5  50.5  1  1.0e-3  1.0e4  0.0  0.0  0.0
6 ...
```

Leaf-cell coordinates are given with the box origin at the corner, [0, BOXLEN]. The reader internally re-centers to $[-L/2, +L/2]$; *do not* pre-shift the data.

7.3 Tau Normalization Convention

For AMR, `par%taumax` is the line-center optical depth from the box center to the $+z$ face along the z -axis. This matches the Cartesian sphere convention where `par%taumax` is the center-to-surface optical depth, provided $r_{\max} = L/2$.

7.4 Source Geometries in AMR Mode

In v2.00, AMR supports 'point' and 'uniform'. In v2.10, most Cartesian geometries (Gaussian, exponential, Sérsic, `star_file`, etc.) are also available; refer to §5.1 and the v2.10 release notes.

8. Clumpy Medium Mode

8.1 Overview

Clumpy medium mode places N non-overlapping spherical clumps of radius r_c inside a spherical outer domain of radius R . The inter-clump medium is vacuum. All opacity resides inside the clumps. Clump positions are drawn by Random Sequential Adsorption (RSA).

8.2 Enabling Clump Mode

v2.00:

```
par%use_clump_medium = .true.
```

v2.10 (preferred):

```
par%grid_type = 'clump'
```

8.3 Clump Parameters

Parameter	Default	Description
<code>par%rmax</code>	1.0	Outer sphere radius (code units)
<code>par%rmin</code>	-999	Inner cavity radius (the default sentinel is treated as 0). Clumps are placed only in the shell $r_{\min} \leq r \leq r_{\max}$
<code>par%clump_radius</code>	—	Individual clump radius (same units)
<code>par%clump_tau0</code>	—	Line-center τ_0 from clump center to its surface
<code>par%clump_f_cov</code>	—	Covering factor $C_f = Nr_c^2/R^2$
<code>par%clump_f_vol</code>	—	Volume filling factor $f_V = N(r_c/R)^3$
<code>par%clump_N_clumps</code>	—	Total number of clumps N
<code>par%temperature</code>	10^4 K	Clump gas temperature
<code>par%clump_sigma_v</code>	0.0	Gaussian σ of random bulk clump velocities [km s^{-1}]
<code>par%save_clump_info</code>	<code>.false.</code>	Write clump positions and velocities to a FITS binary table
<code>par%clump_input_file</code>	"	If non-empty, load the clump population from this FITS file (see §8.5) instead of generating it internally
<code>par%clump_fully_inside</code>	<code>.true.</code>	If <code>.true.</code> (default), reject any candidate whose outermost edge would protrude past the outer sphere ($d_{\text{center}} + r_c > R$) or into the inner cavity ($d_{\text{center}} - r_c < r_{\min}$); if <code>.false.</code> , restore the legacy behavior in which only the center is required to lie in $[r_{\min}, R]$

Specify exactly one of `par%clump_f_cov`, `par%clump_f_vol`, or `par%clump_N_clumps`.

For the per-clump opacity, specify *one* of `par%clump_tau0`, `par%clump_NHI`, or `par%clump_nH`. As a convenience, the system-level radial-sightline targets `par%taumax` and `par%N_HI_max` are also accepted: when none of `par%clump_tau0`/`par%clump_NHI`/`par%clump_nH` is set, LaRT back-solves the peak per-clump opacity so the realised radial sightline from the box centre to $r = R$ matches the requested `par%taumax` (or `par%N_HI_max`). In the uniform-clump case the back-solve uses the closed-form $\tau_{\max} = \frac{4}{3} f_{\text{cov}} \tau_{c,0}$ (and similarly for $N_{\text{HI,max}}$); in profile mode it uses the radial quadrature derived from `par%clump_radius_profile`, `par%clump_density_profile`, and `par%clump_number_profile`. `par%N_gas_max` is treated as an alias for `par%N_HI_max` in clump mode (HI-only line). When a pre-built clump population is loaded via `par%clump_input_file`, supplying `par%taumax` or `par%N_HI_max` multiplies every `cl_rho_kap(i)` by a single factor α so the realised target is matched (small-angle expected-column formula); the per-clump table is otherwise preserved.

When `par%rmin` is set, `par%clump_f_vol` and `par%clump_f_cov` are interpreted relative to the shell volume $V_{\text{shell}} = \frac{4}{3}\pi(R^3 - r_{\min}^3)$ and the shell sightline factor $R^2 + Rr_{\min} + r_{\min}^2$. With `par%rmin` unset (default -999,

treated as 0) these reduce exactly to the original $(R/r_c)^3$ and $(R/r_c)^2$ formulas, preserving prior runs. When `par%clump_fully_inside` is on, the realized filling factors fall slightly below the requested target whenever r_c is a non-trivial fraction of $R - r_{\min}$, since the available placement volume shrinks from $\frac{4}{3}\pi(R^3 - r_{\min}^3)$ to $\frac{4}{3}\pi[(R - r_c)^3 - (r_{\min} + r_c)^3]$. Specify a slightly larger target if exact matching is required. See the *Clumpy Medium Algorithm Description* document, “Fully-Inside Constraint” subsection, for the placement details.

8.4 Spatially-varying clump properties

Three optional radial-profile inputs let the per-clump radius, internal H I density, and spatial number density vary with distance from the box center. Built-in shapes are ‘constant’ (default), ‘powerlaw’, ‘gaussian’, ‘exponential’, and ‘file’ (5-column ASCII table). Setting any flag to a value other than ‘constant’ switches `par%init_clumps` to the profile-aware path.

Parameter	Default	Description
<code>par%clump_radius_profile</code>	‘constant’	shape of $r_c(r)$
<code>par%clump_radius_alpha</code>	0.0	power-law index for $r_c(r)$
<code>par%clump_radius_r0</code>	0.0	power-law scale for $r_c(r)$
<code>par%clump_density_profile</code>	‘constant’	shape of internal $n_H(r)$
<code>par%clump_density_alpha</code>	0.0	power-law index for $n_H(r)$
<code>par%clump_density_r0</code>	0.0	power-law scale for $n_H(r)$
<code>par%clump_number_profile</code>	‘constant’	shape of spatial $n_{cl}(r)$
<code>par%clump_number_alpha</code>	0.0	power-law index for $n_{cl}(r)$
<code>par%clump_number_r0</code>	0.0	power-law scale for $n_{cl}(r)$
<code>par%clump_profile_file</code>	”	5-column tabulated profile (used when any axis = ‘file’)

For non-uniform profiles the closed-form $f_{\text{cov}} / f_{\text{vol}} / N$ relation does not hold. Instead the profile-aware quadrature back-solves the spatial normalization so that whichever of `par%clump_f_cov`, `par%clump_f_vol`, or `par%clump_N_clumps` is provided is matched, and the other two are reported on stdout. See the *Clumpy Medium Algorithm Description* document for the integral definitions and the implementation outline.

8.5 Standalone clump generator (make_clumps.x)

The same Fortran routines that build the clump population inside LaRT are also exposed through a small standalone driver, `make_clumps.x`:

```

1 make make_clumps                                # builds make_clumps.x
2 mpirun -np N ./make_clumps.x clumps.in          # writes <base>_clumps.fits.gz
```

The driver reads the same namelist as `LaRT.x` and writes the realized clump population to `<base_name>_clumps.fits.gz` without running the photon transport. Fix `par%iseed` to a non-zero value for reproducibility: under the same seed the table produced by `make_clumps.x` matches the `_clumps.fits.gz` written by a regular `LaRT.x` run with `par%save_clump_info = .true.` byte-for-byte.

To consume the file, set `par%clump_input_file` on the LaRT input:

```

1 &parameters
2   par%use_clump_medium = .true.
3   par%rmax              = 1.0
4   par%clump_radius      = 0.01
5   par%clump_tau0        = 1.0e3
6   par%temperature       = 1.0e4
7   par%no_photons        = 1e6
8   par%clump_input_file  = 'clumps_clumps.fits.gz'
9 /

```

When `par%clump_input_file` is non-empty, `init_clumps` skips the profile-detection and RSA-generation paths entirely and reads the population from the file. `read_clumps_fits` accepts files with any subset of the per-clump columns `R_CLUMP/RHOKAP/TEMP`; missing columns are filled from the corresponding header keyword (`CL_RAD/RHOKAP/TEMP_CL`). For the `RHOKAP` slot the column names `DENSITY` and `DENS` are also accepted as aliases, in priority order `RHOKAP` → `DENSITY` → `DENS`.

8.6 Clump Velocity

A systematic flow can be added on top of the random component (`par%clump_sigma_v`) via `par%velocity_type`. Supported values in clump mode: `'hubble'`, `'constant_radial'`, `'parallel_velocity'`, `'ssh'`, `'rotating_solid_body'`, `'rotating_galaxy_halo'`.

9. Observers and Peel-off

The peel-off (peeling-off) technique shoots a photon toward the observer at every scattering event, allowing spectral images to be constructed without noise. It is activated by setting `par%nxim` and `par%nyim` to non-zero values.

9.1 Observer Placement

Direction vector:

```

1 par%obsx = 0.0, par%obsy = 0.0, par%obsz = 1.0  ! look down +z axis
2 par%distance = 100.0                          ! observer-center separation

```


Spherical angles:

```

1 par%alpha = 45.0    ! azimuthal angle [deg]
2 par%beta  = 30.0    ! elevation angle [deg]
3 par%distance = 100.0

```

par%gamma sets the rotation of the detector plane about the line of sight. If par%distance is omitted it defaults to $100 \times r_{\max}$ (or the maximum box half-extent).

9.2 Multiple Observers

Multiple observers are specified as arrays:

```

1 par%alpha = 0.0  45.0  90.0  135.0
2 par%beta  = 0.0   0.0   0.0   0.0

```

Up to 99 simultaneous observers are supported.

9.3 Image Parameters

Parameter	Default	Description
par%nxim, par%nyim	0	Image pixel counts; peel-off enabled when > 0
par%dxim, par%dyim	auto	Pixel angular size [deg] (defaults to cover the full grid)

When par%dxim/par%dyim are not set, the pixel size is chosen to cover the full sphere:

$$\delta\theta = \frac{1}{N_x/2} \arcsin\left(\frac{r_{\max}}{d}\right). \quad (3)$$

9.4 Peel-off Output Flags

Parameter	Default	Description
par%save_peeloff	.false.	Save 1-D peel-off spectra per observer
par%save_peeloff_2D	.false.	Save peel-off spectral image cubes (x - y - ν)
par%save_peeloff_3D	.false.	Save full 3-D peel-off data
par%save_sightline_tau	.false.	Compute deterministic sight-line maps from each observer pixel: gas $\tau(\nu)$, N_{HI} , and dust τ (when par%DGR > 0). See §9.5.

9.5 Sight-line Optical-depth and Column-density Maps

When par%save_sightline_tau = .true., LaRT integrates the opacity along each observer pixel's line of sight and writes a separate FITS file <base>_tau<obs>.fits.gz for every observer. The file contains:

HDU	Content
TAU_gas	gas optical depth $\tau(\nu, x, y)$
N_gas	gas column density $N_{\text{HI}}(x, y)$
TAU_dust	dust optical depth $\tau_{\text{dust}}(x, y)$ (only when <code>par%DGR > 0</code>)

The pixel grid uses the same WCS as the peel-off image (`par%nxim, par%nyim, par%dxim, par%dyim`); the frequency axis matches the spectrum output (`par%nxfreq, par%velocity_min, par%velocity_max`). All three grid modes (Cartesian, AMR, clump) are supported with the appropriate raytrace dispatch.

Standalone driver `make_sightline_tau.x`

The same sight-line integration is exposed as a standalone executable that skips the photon-transport stage entirely:

```

1 make sightline_tau                                # builds make_sightline_tau.x
2 mpirun -np N ./make_sightline_tau.x sl.in          # writes <base>_tau.fits.gz
```

`sl.in` is an ordinary LaRT input file; the driver internally sets `par%save_peeloff` and the sight-line flag to `.true.` and otherwise honors every namelist parameter. Useful for parameter sweeps where only the deterministic maps are needed (no Monte-Carlo transport). Unlike `make_clumps.x`, multiple MPI ranks do speed this driver up: pixels are divided across `par%nproc` via `loop_divide` and reduced with `reduce_mem`.

The folder `examples/sightline_tau/` contains a reference set of six runs that exercise every grid mode and observer configuration the driver supports:

File	Grid mode	Observer
<code>car_sightline.in</code>	Cartesian	outside (rectangular FITS)
<code>amr_sightline.in</code>	AMR	outside (rectangular FITS)
<code>clump_sightline.in</code>	clumpy sphere	outside (rectangular FITS)
<code>car_inside.in</code>	Cartesian	inside (HEALPix all-sky)
<code>amr_inside.in</code>	AMR	inside (HEALPix all-sky)

The shipped `run.sh` executes all six in sequence; `make_amr_sphere_data.py` regenerates the AMR data file from a uniform-sphere physics model. The companion notebook `plot_sightline_tau.ipynb` displays the column-density maps, selected $\tau(x, y, \nu)$ slices, the all-sky HEALPix mollview maps and the sky-mean line profile.

tau_huge cap (sight-line vs. photon transport). The AMR raytrace routines used by the photon-transport stage early-exit when the running optical depth exceeds the double-precision $\exp(-\tau)$ underflow point ($\tau_{\text{huge}} \approx 745.2$); accumulating further contributes nothing to the photon's transmission. The sight-line variant `raytrace_to_edge_tau_g`

exists precisely to *report* the integrated τ , however large, and therefore runs without that cap (matching the Cartesian `raytrace_to_edge_tau_gas` in `raytrace_car.f90`). The same applies to `raytrace_to_dist_tau_gas_amr` used by the inside-observer HEALPix path.

9.6 Inside Observer (HEALPix All-Sky)

Setting `par%inside > 0` switches the entire output and peel-off chain to the *inside-observer* path: the observer is placed at a chosen position (o_x, o_y, o_z) *inside* the medium, and each escaping photon is binned into an all-sky HEALPix map by the direction from the observer back to the photon. This mode is intended for “galactic” observations – e.g., placing the observer at the Sun’s position inside a model of the Galactic ISM, or inside a simulated galaxy.

Activation:

```

1 par%inside = 8                ! HEALPix Nside parameter; total npix = 12*inside^2
2 par%obsx   = 0.3              ! observer position (code units)
3 par%obsy   = 0.0
4 par%obsz   = 0.0
5 par%save_peeloff = .true.
6 par%save_peeloff_2D = .true. ! also save the 2D map (sum over frequency)

```

When `par%inside > 0`, `setup.f90` automatically sets `par%observer_located_inside = .true.` and switches:

- the observer arrays from rectangular images to HEALPix all-sky maps with $N_{\text{pix}} = 12N_{\text{side}}^2$;
- the peel-off routines to the `_inside` variants (which use `vec2pix` to bin the peeled weight by observer-pointing direction);
- `output_reduce`, `output_normalize` and `write_output` to the `_inside` variants.

In v2.10 the dispatch is performed by the type-bound methods of `grid_base_type` (`grid_class.f90`); the `grid_car_type` and `grid_amr_type` concrete types each branch on `par%observer_located_inside` and forward to the appropriate `_inside` or `_outside` concrete routine.

Multiple observers. As with the rectangular path, up to 99 observer positions are supported by passing arrays:

```

1 par%inside = 8
2 par%obsx   = 0.0  0.3  0.0
3 par%obsy   = 0.0  0.0  0.5
4 par%obsz   = 0.0  0.0  0.0

```

Output files. In inside-observer mode the `_obs.fits.gz` file holds 2D HEALPix arrays of shape `(npix, nxfreq)` with extensions `Direct` (unscattered direct contribution) and `Scattered` (scattered contribution).

When `par%save_peeloff_2D = .true.` a separate `_obs2D.fits.gz` file is written with shape `(npix,)` holding the same arrays summed over frequency. The HEALPix maps use the RING ordering convention. The Spectrum HDU (`Jout`, `Jin`, `Jabs`) is always written regardless of observer mode.

Source scope. The inside-observer path supports the same internal sources as the standard path – `point`, `uniform`, `gaussian`, `exponential`, `sersic`, `diffuse_emissivity` – but *not* the external-illumination geometries (`point_illumination`, `stellar_illumination`, `plane_illumination`), which are only physically meaningful for an observer outside the medium. Stokes polarization is not yet implemented for the inside-observer path.

Grid-mode support. The HEALPix inside-observer path is available for both Cartesian (`par%grid_type = 'car'`) and AMR (`par%grid_type = 'amr'`) grids. In AMR mode, dedicated `_inside_amr` peeling, output and sightline routines are used (see the *LaRT AMR Description* document for implementation details). Clumpy medium (`par%grid_type = 'clump'`) is not currently supported with the inside-observer path.

Sightline-tau output. When the inside-observer path is active, an additional `_sightline.fits.gz` file is produced that records τ and $N(\text{HI})$ from the observer position to each of the six box faces ($\pm x$, $\pm y$, $\pm z$). This is useful for cross-checking the optical depth structure seen by the observer.

Smoke test. The `examples/amr_sphere_generic/` directory ships `sphere_amr_inside_test.in` (AMR uniform sphere) and `sphere_car_inside_test.in` (Cartesian 64^3 counterpart), both with `par%nside = 8` and an off-center observer at $(o_x, o_y, o_z) = (0.3, 0, 0)$. These produce equivalent HEALPix maps within the documented AMR octree volume-averaging tolerance.

10. Output Files

All outputs are gzip-compressed FITS files (`.fits.gz`).

10.1 Primary Output Arrays

Array	FITS extension	Description
<code>Jout</code>	<code>JOUT</code>	Emergent spectrum integrated over all escape directions
<code>Jin</code>	<code>JIN</code>	Injected (intrinsic) source spectrum
<code>Jabs</code>	<code>JABS</code>	Spectrum of photons absorbed by dust
<code>Jmu</code>	<code>JMU</code>	Emergent spectrum binned by $\mu = \cos \theta_z$

Jout is the main science output. It is normalized per unit frequency interval and per unit area of the bounding sphere. In dimensionless units (no `par%distance_unit`):

$$J_{\text{out}}(x) = \frac{\text{escaped photon weight in bin } x}{n_{\text{ph}} \Delta x 2\pi \pi R^2}, \quad (4)$$

where $R = r_{\text{max}}$ (Cartesian) or $R = L_{\text{box}}/2$ (AMR), and $\Delta x = 1$ in units of the Doppler width.

Jin requires `par%save_Jin = .true.`

Jabs requires `par%save_Jabs = .true.`

Jmu requires `par%save_Jmu = .true.`; see §10.2.

The total flux is conserved: $\int J_{\text{out}} dx + \int J_{\text{abs}} dx = \int J_{\text{in}} dx$ (in the absence of other losses).

10.2 Angular Spectrum Jmu

Setting `par%save_Jmu = .true.` adds a 2-D image extension `EXTNAME='Jmu'` of shape (n_x, n_μ) that records the emergent spectrum binned by the z -component of the escape direction, $\mu \equiv \cos \theta_z = \hat{k}_z$. This is useful for

- verifying isotropy of homogeneous models (every μ bin should match J_{out} within Poisson noise),
- characterizing the angular dependence of escape in non-isotropic models (for example, a slab or rotating halo).

Binning.

- `par%nmu` bins of width $\Delta\mu$ tile the μ range uniformly.
- Default `par%nmu = 11` (odd) places $\mu = 0$ at a bin center, so the perpendicular-escape spectrum is preserved as a single channel. Even `par%nmu` places $\mu = 0$ at a bin edge and cleanly separates the $+z$ and $-z$ hemispheres.
- Range:
 - `par%xyz_symmetry = .true.` folds escapes onto the $+z$ hemisphere, so $\mu \in [0, +1]$ and $\Delta\mu = 1/n_\mu$.
 - Otherwise $\mu \in [-1, +1]$ and $\Delta\mu = 2/n_\mu$.

The header keywords `nmu`, `mu_min`, `dmu` record these values, and full WCS keywords (`CTYPE1='XFREQ'`, `CTYPE2='MU'`, `CRVAL*`, `CDELTA*`) are written to the image header.

Normalization. `Jmu` uses the same denominator as `Jout` multiplied by n_μ , so that

$$\frac{1}{n_\mu} \sum_{i=1}^{n_\mu} J(x; \mu_i) = J_{\text{out}}(x) \quad (5)$$

holds exactly. Equivalently, in a homogeneous, isotropic medium each μ bin reproduces `Jout` up to Poisson fluctuations, providing a cheap regression test for new geometries.

Modes. Available with all three grid modes: Cartesian, AMR octree, and the clumpy medium.

10.3 Peel-off cube units and conversion to Jmu units

When peel-off is enabled (`par%nxim, par%nyim > 0` and `par%save_peeloff_3D = .true.`), each observer's `<base>_obs_NNN.fits.gz` file contains `Scattered` and `Direct` 3-D image cubes, plus optionally `Direct0`. These are written by `output_sum_rect.f90` as a *specific intensity per pixel*, normalized so that

$$I_{\text{peel}}(x, y, \nu) = \frac{\text{photon weight in pixel } (x, y) \text{ and bin } \nu}{n_{\text{ph}} \delta\Omega_{\text{pix}} \Delta\nu d_{\text{cm}}^2}, \quad (6)$$

where $\delta\Omega_{\text{pix}} = (\text{CD2_2} \cdot \text{CD3_3}) (\pi/180)^2$ is the per-pixel solid angle, $\Delta\nu$ is `Dxfreq` or `Dwave` (set by `par%intensity_unit`), d_{cm}^2 is the square of the distance unit in cm, and n_{ph} is the photon count.

Conversion to Jmu units. `Jout` and `Jmu` are normalized by $4\pi R^2$ (the source surface area) and the outward solid angle 2π , whereas the peel-off cube is normalized by the per-pixel solid angle and d_{cm}^2 . Summing the cube over pixels and rescaling brings the two onto a common axis:

$$J_{\text{peel}}(\nu) = \left(\sum_{(x,y)} I_{\text{peel}}(x, y, \nu) \right) \cdot \frac{4\pi D^2 \delta\Omega_{\text{pix}}}{A_{\text{surface}} \cdot 2\pi}, \quad (7)$$

where D is `DISTANCE` (in code units) and A_{surface} is the source surface area in code units, with the geometry-dependent forms mirroring `output_sum_rect.f90`:

Geometry	A_{surface} (code units)	Notes
sphere	$4\pi r_{\text{max}}^2$	default; clumpy medium also uses this
box (rectangle)	$8(x_{\text{max}} y_{\text{max}} + y_{\text{max}} z_{\text{max}} + z_{\text{max}} x_{\text{max}})$	full surface of $[-x_{\text{max}}, x_{\text{max}}] \times \dots$ box
slab (<code>par%xy_periodic</code>)	$2(2x_{\text{max}})(2y_{\text{max}})$	two unbounded boundaries
AMR octree	4π	equivalent to $r_{\text{max}} = 1$ (cf. §7.)

The bin width $\Delta\nu$ cancels because both `Jmu` and `I_peel` are already normalized per bin. At convergence and for an axisymmetric source,

$$J_{\text{peel}}(\nu; \alpha, \beta) \rightarrow J_{\text{mu}}(\nu; \mu = \cos \beta). \quad (8)$$

The Python helper `LaRTOutput.plot_peel_jmu_compare()` (in `python/read_lart.py`) implements Eq. (7) with the surface area chosen automatically from `par%geometry`, `par%rmax`, and the spectrum-header `XMAX/YMAX/ZMAX/XY_PER`. See also `examples/sphere/plot_Jpeel_conversion.ipynb` and `examples/clump_sphere/verify_peel_jmu.ipynb`.

Component decomposition (clumpy medium). For a point source inside a clumpy sphere with covering factor f_{cov} , the fraction of photons that escapes without scattering is $\exp(-f_{\text{cov}})$ (Poisson statistics), and the fraction that scatters at least once is $1 - \exp(-f_{\text{cov}})$. `Jmu` tallies *both* populations. The peel-off cube splits them:

- `Scattered`. The MC estimator over many scatter events averages efficiently, so

$$\int J_{\text{peel}}^{\text{scatt}} d\nu \approx (1 - e^{-f_{\text{cov}}}) \int J_{\text{mu}} d\nu, \quad (9)$$

and the equality holds across all observer angles within Monte-Carlo noise.

- **Direct.** For a *point* source, every photon shares the same line of sight to a given observer, so the direct contribution is the binary realization of $\exp(-\tau_{\text{LOS}})$: either ~ 1 (clear LOS) or ~ 0 (blocked by an intervening clump) per observer. J_{mu} , by contrast, is azimuth averaged and captures direct escapes uniformly. Adding the binary **Direct** and the smooth **Scattered** therefore only matches J_{mu} *in expectation* – it can deviate substantially per observer at finite photon counts.

For shape comparisons in clumpy runs, prefer the scattered component (`component='scatt'` in `plot_peel_jmu_compare` so that Eq. (9) provides a clean, deterministic relation.

10.4 Output Control Parameters

Parameter	Default	Description
<code>par%out_file</code>	<code>"</code>	Output file name (defaults to <code><input>.fits.gz</code>)
<code>par%out_merge</code>	<code>.false.</code>	Accumulate into existing output instead of overwriting
<code>par%out_bitpix</code>	<code>-32</code>	FITS precision: <code>-32 = float32</code> , <code>-64 = float64</code>
<code>par%save_Jin</code>	<code>.false.</code>	Save injected spectrum
<code>par%save_Jabs</code>	<code>.false.</code>	Save dust-absorbed spectrum
<code>par%save_Jmu</code>	<code>.false.</code>	Save angular spectrum $J(x; \mu)$ (§10.2)
<code>par%nmu</code>	<code>11</code>	Number of μ bins for J_{mu}
<code>par%save_all</code>	<code>.false.</code>	Save full 3-D $J(\nu, x, y, z)$ array
<code>par%save_direc0</code>	<code>.false.</code>	Save photon escape direction distributions
<code>par%save_radial_profile</code>	<code>.false.</code>	Save 1-D radial J profile
<code>par%save_sightline_tau</code>	<code>.false.</code>	Save optical depth along sightlines
<code>par%intensity_unit</code>	<code>"</code>	Physical unit label written to the FITS header

`par%out_merge = .true.` is useful for noise reduction: run the same setup multiple times and merge the results. All input parameters must be identical across runs.

11. Dust and Polarization

Parameter	Default	Description
<code>par%DGR</code>	0.0	Dust-to-gas ratio relative to the Milky Way value
<code>par%albedo</code>	0.32536	Single-scattering albedo ω
<code>par%hgg</code>	0.67617	Henye–Greenstein asymmetry parameter g
<code>par%use_stokes</code>	<code>.false.</code>	Enable full Stokes polarization RT
<code>par%scatt_mat_file</code>	"	Mueller matrix file for polarized dust scattering
<code>par%use_reduced_wgt</code>	<code>.false.</code>	Reduce photon weight by albedo instead of destroying absorbed photons

When `par%DGR = 0` (default), there is no dust and all photon weight is conserved. Non-zero `par%DGR` adds dust extinction and Henye–Greenstein scattering. When `par%use_stokes = .true.`, a Mueller matrix file must be provided; files for the Weingartner–Draine Milky Way model are included in the `data/` directory.

12. Parallelism

Parameter	Default	Description
<code>par%no_photons</code>	10^6	Total photon packets (floating-point, converted to int64)
<code>par%use_master_slave</code>	<code>.true.</code>	Dynamic load balancing via master–worker algorithm
<code>par%num_send_at_once</code>	100	Photons dispatched to each worker per batch
<code>par%nprint</code>	10^5	Progress-report interval [photons]
<code>par%iseed</code>	auto	Random seed; defaults derived from MPI rank and system clock

With `par%use_master_slave = .true.`, one MPI rank acts as dispatcher and the rest as workers. This balances workloads when scattering rates vary across photon packets. With `.false.`, each rank processes an equal share of `par%no_photons`.

13. Complete Parameter Reference

Table 1 lists commonly used parameters. Parameters marked † are new or changed in v2.10.

Table 1: LaRT parameter reference.

Parameter	Default	Description
<i>Simulation control</i>		
<code>par%no_photons</code>	10^6	Number of photon packets
<code>par%nprint</code>	10^5	Progress interval [photons]
<code>par%iseed</code>	auto	Random seed

continued on next page

(continued)

Parameter	Default	Description
<i>Cartesian grid</i>		
par%nx, par%ny, par%nz	101	Cells per axis
par%xmax, par%ymax, par%zmax	1.0	Half-box size (code units)
par%rmax	-999	Spherical boundary radius
par%rmin	-999	Inner cavity radius
par%geometry	'sphere'	J output geometry (§4.2)
par%xyz_symmetry	.false.	+x, +y, +z octant only
par%xy_symmetry	.false.	+x, +y quadrant only
par%xy_periodic	.false.	Periodic in x, y
par%distance_unit	"	Physical unit ('kpc', 'pc', 'au')
par%distance2cm	1.0	1 code unit in cm
<i>Grid mode</i>		
par%grid_type†	'car'	v2.10+: 'car', 'amr', 'clump'
par%use_amr_grid	.false.	v2.00 AMR flag
par%use_clump_medium	.false.	v2.00 clump flag
<i>Source</i>		
par%source_geometry	'point'	See §5.1
par%xs_point, par%ys_point, par%zs_point	0.0	Point source position
par%source_rmax	-999	Source truncation radius
par%source_rscale	-999	Radial scale length
par%source_zscale	0.0	Vertical scale height
par%sersic_m	4.0	Sérsic index
par%Reff	1.0	Effective radius
par%comoving_source	.true.	Source co-moving with local gas
par%emiss_file	"	Emissivity FITS cube
par%star_file	"	Point-star list (x, y, z, L)
<i>Line and spectrum</i>		
par%line_id	$\text{Ly}\alpha$	Atomic transition (§5.3)
par%spectral_type	'monochromatic'	Emission frequency distribution
par%temperature	10^4 K	Gas temperature

continued on next page

(continued)

Parameter	Default	Description
par%temperature0	$= T$	Source temperature for voigt0
par%bturb	0.0	Turbulent dispersion [km s^{-1}]
par%gaussian_width_vel	0.0	Gaussian line width [km s^{-1}]
par%xfreq0	0.0	Frequency offset for emitted photons
par%line_prof_file	"	Custom line profile file
par%line_prof_file_type	0	0: (x, I) ; 1: (λ, I)
<i>Frequency grid</i>		
par%nxfreq	121	Spectral bins
par%xfreq_min,	auto	x -grid limits
par%xfreq_max		
par%nwavelength	0	Wavelength bins (overrides x -grid)
par%wavelength_min,	—	[Å]
par%wavelength_max		
par%nvelocity	0	Velocity bins (overrides x -grid)
par%velocity_min,	—	[km s^{-1}]
par%velocity_max		
<i>Optical depth and density</i>		
par%taumax	—	Line-center τ_0 : center $\rightarrow +z$ boundary
par%tauhomo	—	Same, uniform-density equivalent
par%N_HI_max	—	Column density: center $\rightarrow +z$ [cm^{-2}]
par%N_HI_homo	—	Same, uniform-density equivalent
par%density_rscale	-999	Exponential density scale radius
<i>External data</i>		
par%dens_file	"	Density FITS cube [cm^{-3}]
par%temp_file	"	Temperature FITS cube [K]
par%velo_file	"	Velocity FITS cube [km s^{-1}]
par%input_field	"	Base name for .dens/.temp/.velo.fits.gz
<i>Velocity</i>		
par%velocity_type	"	Velocity law (§6.4)
par%Vexp	0.0	Expansion/infall speed [km s^{-1}]

continued on next page

(continued)

Parameter	Default	Description
par%Vx, par%Vy, par%Vz	0.0	Uniform velocity [km s^{-1}]
par%Vpeak	0.0	Peak velocity for SSH [km s^{-1}]
par%rpeak	0.0	Radius at peak for SSH
par%DeltaV	0.0	Velocity increment for SSH [km s^{-1}]
par%Vrot	0.0	Rotation speed [km s^{-1}]
par%recoil	.false.	Photon recoil in scattering
<i>AMR</i>		
par%amr_type	'ramses'	'generic' or 'ramses'
par%amr_file	"	AMR data path
par%amr_snapnum	1	RAMSES snapshot number
<i>Clumpy medium</i>		
par%clump_radius	—	Clump radius
par%clump_tau0	—	Line-center τ_0 per clump
par%clump_f_cov	—	Covering factor C_f
par%clump_f_vol	—	Volume filling factor f_v
par%clump_N_clumps	—	Number of clumps N
par%clump_sigma_v	0.0	Random velocity dispersion [km s^{-1}]
par%save_clump_info	.false.	Write clump table to FITS
<i>Observers and peel-off</i>		
par%nxim, par%nyim	0	Image pixels; peel-off active when > 0
par%dxim, par%dyim	auto	Pixel size [deg]
par%distance	$100r_{\text{max}}$	Observer–center distance
par%obsx, par%obsy, par%obsz	(0,0,1)	Observer direction
par%alpha, par%beta	—	Azimuth, elevation [deg]
par%gamma	0.0	Detector-plane rotation [deg]
par%save_peeloff	.false.	1-D peel-off spectra
par%save_peeloff_2D	.false.	Peel-off spectral images
par%save_peeloff_3D	.false.	Full 3-D peel-off cube
par%save_direct0	.false.	Peel-off image without $e^{-\tau}$ (transparent-medium reference)
<i>Output</i>		

continued on next page

(continued)

Parameter	Default	Description
par%out_file	"	Output FITS file name
par%out_merge	.false.	Accumulate into existing file
par%out_bitpix	-32	FITS precision
par%save_Jin	.false.	Save injected spectrum
par%save_Jabs	.false.	Save absorbed spectrum
par%save_Jmu	.false.	Save angular spectrum $J(x; \mu)$
par%nmu	11	Number of μ bins for J_{μ}
par%save_all	.false.	Save full 3-D $J(\nu, x, y, z)$
par%save_radial_profile	.false.	Save 1-D radial J
par%save_sightline_tau	.false.	Save sightline τ
par%intensity_unit	"	FITS header unit label
<i>Dust</i>		
par%DGR	0.0	Dust-to-gas ratio
par%albedo	0.32536	Single-scattering albedo
par%hgg	0.67617	Henyey–Greenstein asymmetry g
par%use_stokes	.false.	Polarization RT
par%scatt_mat_file	"	Mueller matrix file
par%use_reduced_wgt	.false.	Reduce weight instead of destroy
<i>Parallelism</i>		
par%use_master_slave	.true.	Dynamic load balancing
par%num_send_at_once	100	Photons per worker batch

14. Python Utilities

A small set of post-processing scripts is shipped under `python/` in each version's source tree. They depend on `numpy`, `astropy`, and (for plotting scripts) `matplotlib`.

14.1 Reading FITS output: `read_lart.py`

`read_lart.py` is a thin reader that loads the spectral arrays from a LaRT FITS output. It accepts the input-file name in any of three forms:

- full input file: `t1tau2.in`

- stem only: `t1tau2` (.in is added automatically)
- the FITS file: `t1tau2.fits.gz`

The script honors `par%out_file` when set in the namelist; otherwise it falls back to `<stem>.fits.gz` in the same directory as the input file.

Module API.

```

1 from read_lart import read_lart
2 out = read_lart('t1tau2.in')
3
4 # 1-D arrays, length nxfreq
5 out.xfreq, out.velocity, out.wavelength
6 out.Jout          # always present
7 out.Jin, out.Jabs, out.Jabs2    # optional -- None if not in file
8
9 # Jmu output -- present only if par%save_Jmu = .true. was used
10 out.mu           # bin centers, length nm_u
11 out.mu_edges     # bin edges, length nm_u+1
12 out.Jmu          # shape (nm_u, nxfreq); None if absent
13 out.nm_u, out.mu_min, out.dmu
14
15 # Raw FITS headers
16 out.spectrum_header, out.jmu_header

```

Every optional array is `None` when its FITS column or HDU is not present, so the reader works on outputs from any LaRT version.

Plotting Jmu. `LaRTOutput.plot_jmu()` draws the angular spectrum directly:

```

1 out = read_lart('t1tau3.in')
2 out.plot_jmu(kind='lines', x='velocity')    # one curve per mu bin
3 out.plot_jmu(kind='image', x='velocity')    # 2-D heatmap (x vs mu)

```

The `x=` keyword selects the abscissa ('velocity', 'xfreq', or 'wavelength'); the `kind=` keyword toggles between line ensemble (colored by μ) and a 2-D image. The line view also overplots `Jout` as a black dashed reference. Pass `ax=` to draw onto an existing axes, `savefig='out.png'` to save to disk. Axis limits accept either the tuple form `xlim=(lo,hi) / ylim=(lo,hi)` or the per-bound shorthand `xmin/xmax/ymin/ymax`; the per-bound override takes precedence when both are given:

```

1 out.plot_jmu(kind='lines', xlim=(-500, 500), ymin=0)
2 out.plot_jmu(kind='lines', xmax=300)          # left edge auto-scaled

```

Command-line usage.

```

1 python read_lart.py t1tau2          # auto-resolves to t1tau2.in
2 python read_lart.py t1tau2.in
3 python read_lart.py t1tau2.fits.gz

```

prints a one-screen summary of which arrays are present and, when `Jmu` is in the file, the per-bin $\max_x J(x; \mu) / \max_x J_{\text{out}}(x)$ ratio (a quick isotropy sanity check for homogeneous models).

Other scripts. The following plotting helpers are also shipped:

- `plot_spectrum.py`, `plot_map.py`, `plot_intensity_profile.py`, `plot_Pa.py`: standard quick-look plots.
- `check_flux.py`, `flux_check.py`: flux-conservation diagnostics.
- `AMR_grid/`: helpers for generating generic AMR data files (see `AMR_grid/make_amr_grid.py`, `make_amr_sphere_radial.py`).

15. Example Input Files

15.1 Uniform Sphere, $\text{Ly}\alpha$, Static

```

1 &parameters
2   par%no_photons      = 1e6
3   par%temperature     = 1.0e4
4   par%taumax          = 1.0e4
5   par%rmax            = 1.0
6   par%nx = 101, par%ny = 101, par%nz = 101
7   par%source_geometry = 'point'
8   par%spectral_type   = 'voigt'
9   par%nxfreq          = 121
10 /

```

15.2 Sphere with Hubble Outflow and Peel-off

```

1 &parameters
2   par%no_photons      = 1e6
3   par%temperature     = 1.0e4
4   par%taumax          = 1.0e4
5   par%rmax            = 1.0
6   par%nx = 101, par%ny = 101, par%nz = 101
7   par%source_geometry = 'point'
8   par%spectral_type   = 'voigt'
9   par%velocity_type   = 'hubble'
10  par%Vexp             = 200.0          ! km/s outflow
11  par%nxfreq           = 121
12  par%nxim = 129, par%nyim = 129
13  par%alpha = 0.0  45.0  90.0
14  par%beta  = 0.0   0.0   0.0
15  par%save_peeloff_2D = .true.
16 /

```

15.3 Realistic Galaxy (FITS Density Field, Stokes Polarization)

```

1 &parameters
2   par%no_photons      = 1e8
3   par%dens_file       = 'galaxy.dens.fits.gz'
4   par%temp_file       = 'galaxy.temp.fits.gz'
5   par%velo_file       = 'galaxy.velo.fits.gz'
6   par%taumax          = 1.0e4          ! rescale to this optical depth
7   par%distance_unit   = 'kpc'
8   par%source_geometry = 'exponential'
9   par%source_rscale   = 3.0
10  par%source_zscale   = 0.5
11  par%spectral_type    = 'voigt'
12  par%DGR              = 1.0
13  par%use_stokes       = .true.
14  par%nxfreq           = 121
15  par%nxim = 200, par%nyim = 200
16  par%save_peeloff_2D = .true.
17 /

```

15.4 AMR Sphere (Generic Text Format)

```

1 &parameters
2   par%use_amr_grid   = .true.           ! or par%grid_type = 'amr' in v2.10
3   par%amr_type       = 'generic'
4   par%amr_file       = 'sphere_amr.dat'
5   par%no_photons     = 1e6
6   par%taumax        = 1.0e4
7   par%source_geometry = 'point'
8   par%spectral_type  = 'voigt'
9   par%nxfreq         = 121
10  par%nxim = 100, par%nyim = 100
11  par%save_peeloff_2D = .true.
12  par%save_Jin       = .true.
13 /

```

15.5 Clumpy Sphere with Random Velocities

```

1 &parameters
2   par%use_clump_medium = .true.         ! or par%grid_type = 'clump' in v2.10
3   par%rmax             = 1.0
4   par%clump_radius     = 0.01
5   par%clump_f_cov      = 5.0
6   par%clump_tau0       = 1.0e3
7   par%temperature      = 1.0e4
8   par%clump_sigma_v    = 50.0           ! km/s random dispersion
9   par%no_photons       = 1e6
10  par%spectral_type     = 'voigt'
11  par%nxfreq           = 301
12  par%velocity_min     = -200.0
13  par%velocity_max     = 200.0
14  par%save_clump_info  = .true.
15 /

```